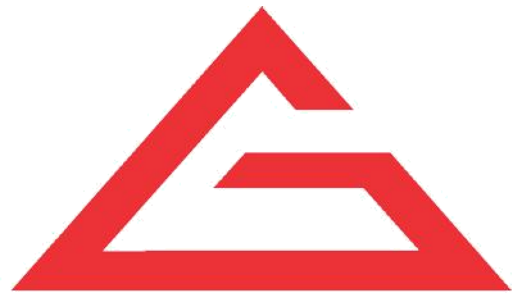


FLUTTER -04

LAB ASSIGNMENTS



G-TEC EDUCATION

— G-TEC Group of Institutions —

www.gteceducation.com

Admin Office House of G-TEC, Calicut-02., India. | Corp. Office Peace Centre, Singapore
— 228149

Add Navigation Rail

Most apps can't fit everything into a single screen. This particular app probably could, but for didactic purposes, you are going to create a separate screen for the user's favorites. To switch between the two screens, you are going to implement your first StatefulWidget.



To get to the meat of this step as soon as possible, split MyHomePage into 2 separate widgets.

Select all of MyHomePage, delete it, and replace with the following code:

[lib/main.dart](#)

// ...

```
class MyHomePage extends StatelessWidget
{
  @override
  Widget build(BuildContext
    context) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            child:
              NavigationRail(
                extended: false,
                destinations: [
                  NavigationRailDestinatio
                    n(
                      icon:
                        Icon(Icons.home),
                      label: Text('Home'),
```

```
class GeneratorPage extends  
StatelessWidget {@override  
Widget build(BuildContext context) {  
  var appState =  
  context.watch<MyAppState>(); var pair =  
  appState.current;
```

IconData icon;

if

(appState.favorites.contains(pair)) {icon = Icons.favorite;

} else {

icon = Icons.favorite_border;

}

return

Center(

child:

Column(

mainAxisAlignment:

MainAxisAlignment.center, children: [

BigCard(pair: pair),

SizedBox(hei

ght: 10), Row(

mainAxisSize:

MainAxisSize.min, children:

[

ElevatedButton.icon

(onPressed: () {

appState.toggleFavorite();

},

icon: Icon(icon),

label: Text('Like'),

),

SizedBox(wid

th: 10),

ElevatedButt

on(

onPressed: () {

```
appState.getNext();  
},  
  
child: Text('Next'),  
),  
),  
),  
),  
),  
),  
},  
}  
}  
  
// ...
```

When saved, you'll see that the visual side of the UI is ready—but it doesn't work. Clicking ♥ (the heart) in the navigation rail does nothing.



Examine the changes.

- First, notice that the entire contents of MyHomePage is extracted into a new widget, GeneratorPage. The only part of the old MyHomePage widget that didn't getextracted is Scaffold.

- The new MyHomePage contains a Row with two children. The first widget is SafeArea, and the second is an Expanded widget.
- The SafeArea ensures that its child is not obscured by a hardware notch or a statusbar. In this app, the widget wraps around NavigationRail to prevent the navigation buttons from being obscured by a mobile status bar, for example.
- You can change the extended: false line in NavigationRail to true. This shows the labels next to the icons. In a future step, you will learn how to do this automatically when the app has enough horizontal space.
- The navigation rail has two destinations (*Home* and *Favorites*), with their respective icons and labels. It also defines the current selectedIndex. A selected index of zero selects the first destination, a selected index of one selects the second destination, and so on. For now, it's hard coded to zero.
- The navigation rail also defines what happens when the user selects one of the destinations with onDestinationSelected. Right now, the app merely outputs the requested index value with print().
- The second child of the Row is the Expanded widget. Expanded widgets are extremely useful in rows and columns—they let you express layouts where some children take only as much space as they need (SafeArea, in this case) and other widgets should take as much of the remaining room as possible (Expanded, in this case). One way to think about Expanded widgets is that they are "greedy". If you want to get a better feel of the role of this widget, try wrapping the SafeArea widget with another Expanded. The resulting layout looks something like this:



- Two Expanded widgets split all the available horizontal space between themselves, even though the navigation rail only really needed a little slice on the left.
- Inside the Expanded widget, there's a colored Container, and inside the container, the GeneratorPage.

Stateless versus stateful widgets

Until now, MyAppState covered all your state needs. That's why all the widgets you have written so far are stateless. They don't contain any mutable state of their own. None of the widgets can change *itself*—they must go through MyAppState.

This is about to change.

You need some way to hold the value of the navigation rail's selectedIndex. You also want to be able to change this value from within the onDestinationSelected callback.

You *could* add selectedIndex as yet another property of MyAppState. And it would work. But you can imagine that the app state would quickly grow beyond reason if every widget stored its values in it.

```
class MyAppState extends ChangeNotifier {  
  var current = WordPair.random();  
  var favorites = <WordPair>[];  
  var selectedIndex = 0;  
  var selectedIndexInAnotherWidget = 0;  
  var indexInYetAnotherWidget = 42;  
  var optionASelected = false;  
  var optionBSelected = false;  
  var loadingFromNetwork = false;  
  // ...  
}
```



Some state is only relevant to a single widget, so it should stay with that widget.

Enter the StatefulWidget, a type of widget that has State. First, convert MyHomePage to a stateful widget.

Place your cursor on the first line of MyHomePage (the one that starts with class MyHomePage...), and call up the **Refactor** menu using Ctrl+. or Cmd+.. Then, select **Convert to StatefulWidget**.

The IDE creates a new class for you, _MyHomePageState. This class extends State, and can therefore manage its own values. (It can change *itself*.) Also notice that the build method from the old, stateless widget has moved to the _MyHomePageState

(instead of staying in the widget). It was moved verbatim—nothing inside the build method changed. It now merely lives somewhere else.

The underscore (_) at the start of `_MyHomePageState` makes that class private and is enforced by the compiler. If you want to know more about privacy in Dart, and other topics, read the [Language Tour](#).

`setState`

The new stateful widget only needs to track one variable: `selectedIndex`. Make the following 3 changes to `_MyHomePageState`:

[lib/main.dart](#)

```
// ...
```

```
class _MyHomePageState extends
```

```
State<MyHomePage> {var selectedIndex = 0; // ←
```

Add this property.

Widget

```
build(BuildContext  
context) { return
```

```
Scaffold(
```

```
body:
```

```
Row(
```

```
children:
```

```
[
```

```
SafeArea
```

```
(
```

```
child:
```

```
NavigationRa
```

```
il( extended:
```

```
false,
```

```
destinations: [
```

```
NavigationRailDe
```

```
stination(icon:
```

Icon(Icons.home)

```
1
label: Text('Home'),
),
NavigationRailDestination(
    icon:
    Icon(Icons.favorite),
    label:
    Text('Favorites'),
),
],
selectedIndex: selectedIndex, //
← Change to this.
onDestinationSelected: (value) {

    // ↓ Replace print with
    this.setState() {
        selectedIndex = value;
    });

1
),
),
Expanded(
    child: Container(
    color:
    Theme.of(context).colorScheme.primaryContainer,
    child: GeneratorPage(),
1
),
1
),
):
```

```
}  
}  
  
// ...
```

Examine the changes:

1. You introduce a new variable, `selectedIndex`, and initialize it to 0.
2. You use this new variable in the `NavigationRail` definition instead of the hard-coded 0 that was there until now.
3. When the `onDestinationSelected` callback is called, instead of merely printing the new value to console, you assign it to `selectedIndex` inside a `setState()` call. This call is similar to the `notifyListeners()` method used previously—it makes sure that the UI updates.

The navigation rail now responds to user interaction. But the expanded area on the right stays the same. That's because the code isn't using `selectedIndex` to determine what screen displays.

Use `selectedIndex`

Place the following code at the top of `_MyHomePageState`'s `build` method, just before `return Scaffold`:

[lib/main.dart](#)

// ...

Widget page;

switch (selectedIndex)

{ case 0:

page =

GeneratorPage();

break;

case 1:

page =

Placeholder();

break;

default:

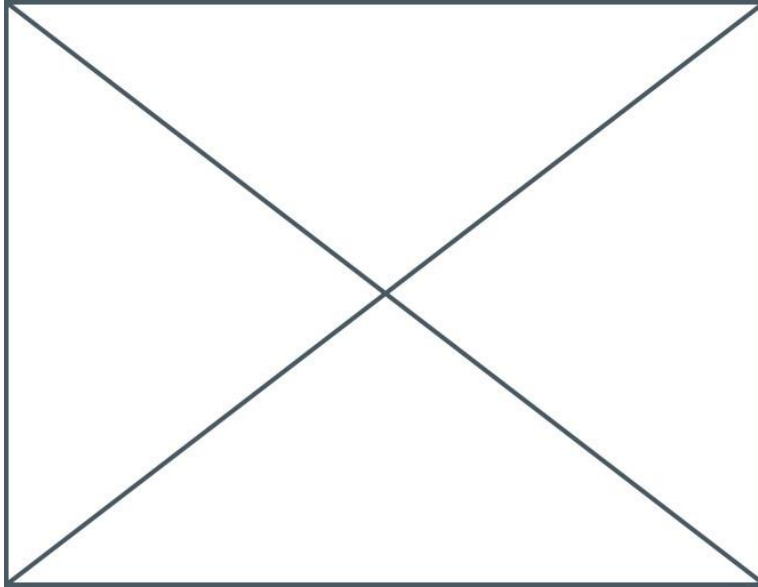
throw UnimplementedError('no widget for \$selectedIndex');

}

//

Examine this piece of code:

1. The code declares a new variable, page, of the type Widget.
2. Then, a switch statement assigns a screen to page, according to the current value in selectedIndex.
3. Since there's no FavoritesPage yet, use Placeholder; a handy widget that draws a crossed rectangle wherever you place it, marking that part of the UI as unfinished.



4. Applying the [fail-fast principle](#), the switch statement also makes sure to throw an error if selectedIndex is neither 0 or 1. This helps prevent bugs down the line. If you ever add a new destination to the navigation rail and forget to update this code, the program crashes in development (as opposed to letting you guess why things don't work, or letting you publish a buggy code into production).

Now that page contains the widget you want to show on the right, you can probably guess what other change is needed.

Here's `_MyHomePageState` after that single remaining change:

[lib/main.dart](#)

```
// ...
```

```
class _MyHomePageState extends  
  State<MyHomePage> {var selectedIndex = 0;  
  
  @override  
  Widget build(BuildContext context) {
```

Widget page;

switch

(selectedIndex) {

case 0:

page =

GeneratorPage()

;break;

case 1:

page =

Placeholder();

break;

default:

throw UnimplementedError('no widget for
\$selectedIndex');

}

return

Scaffold(

body:

Row(

children:

[

SafeArea(

child:

NavigationRa

il(extended:

false,

destinations: [

NavigationRailDe

stination(icon:

Icon(Icons.home)

```

    label: Text('Home'),
  ),

  NavigationRailDestination(
    icon:
      Icon(Icons.favorite),
    label:
      Text('Favorites'),
  ),
],
selectedIndex: selectedIndex,
onDestinationSelected:
  (value) { setState(() {
    selectedIndex = value;
  });
},
),
),
Expanded(
  child: Container(
    color: Theme.of(context).colorScheme.primaryContainer,
    child: page, // ← Here.
  ),
),
),
);
}
}

// ...

```

The app now switches between our GeneratorPage and the placeholder that will soon become the **Favorites** page.

Responsiveness

Next, make the navigation rail responsive. That is to say, make it automatically show the labels (using `extended: true`) when there's enough room for them.



Flutter provides several widgets that help you make your apps *automatically* responsive. Foreexample, Wrap is a widget similar to Row or Column that automatically wraps children to the next "line" (called "run") when there isn't enough vertical or horizontal space.

There's FittedBox, a widget that automatically fits its child into available space according to your specifications.

But NavigationRail doesn't *automatically* show labels when there's enough space because it can't know what *is* enough space in every context. It's up to you, the developer, to make that call.

Say you decide to show labels only if MyHomePage is at least 600 pixels wide.

Note: Flutter works with logical pixels as a unit of length. They are also sometimes called **device-independent pixels**. A padding of 8 pixels is visually the same regardless of whether the app is running on an old low-res phone or a newer 'retina' device. There are roughly 38 logical pixels per centimeter, or about 96 logical pixels per inch, of the physical display.

The widget to use, in this case, is LayoutBuilder. It lets you change your widget tree depending on how much available space you have.

Once again, use Flutter's **Refactor** menu in VS Code to make the required changes. This time, though, it's a little more complicated:

1. Inside `_MyHomePageState`'s build method, put your cursor on Scaffold.
2. Call up the **Refactor** menu with Ctrl+. (Windows/Linux) or Cmd+. (Mac).
3. Select **Wrap with Builder** and press **Enter**.
4. Modify the name of the newly added Builder to LayoutBuilder.
5. Modify the callback parameter list from (context) to (context, constraints).

LayoutBuilder's builder callback is called every time the constraints change. This happens when, for example:

- The user resizes the app's window
- The user rotates their phone from portrait mode to landscape mode, or back
- Some widget next to MyHomePage grows in size, making MyHomePage's constraints smaller
- And so on

Now your code can decide whether to show the label by querying the current constraints. Make the following single-line change to `_MyHomePageState`'s build method:

[lib/main.dart](#)

```
// ...
```

```
class _MyHomePageState extends  
  State<MyHomePage> {var selectedIndex = 0;  
  
  @override
```

Widget

build(BuildContext
context) { Widget page;

switch

(selectedIndex) {

case 0:

page =

GeneratorPage()

; break;

case 1:

page =

Placeholder();

break;

default:

throw UnimplementedError('no widget for
\$selectedIndex');

}

return LayoutBuilder(builder:
(context, constraints) { return
Scaffold(

body:

Row(

children:

[

SafeArea

(

child: NavigationRail(

```

extended: constraints.maxWidth >= 600, // ←
Here.destinations: [
  NavigationRailDe
  stination(icon:
  Icon(Icons.home)

  {
    label: Text('Home'),
  },
  NavigationRailDestinati
  on(          icon:
    Icon(Icons.favorite),
    label:
      Text('Favorites'),
  ),
],
selectedIndex: selectedIndex,
onDestinationSelected:
  (value) { setState(() {
    selectedIndex = value;
  });
},
),

```

```
),  
Expanded(  
    child: Container(  
        color: Theme.of(context).colorScheme.primaryContainer,  
        child: page,  
    ),  
),  
],  
),  
);  
});  
}  
}  
  
// ...
```

Now, your app responds to its environment, such as screen size, orientation, and platform. In other words, it's responsive.

The only work that remains is to replace that Placeholder with an actual **Favorites** screen.